

Rahul Ray

wirelessProject_report

 Reports

Document Details

Submission ID

trn:oid:::3618:138473845

Submission Date

May 11, 2026, 10:42 AM GMT+5:30

Download Date

May 11, 2026, 12:14 PM GMT+5:30

File Name

wirelessProject_report.pdf

File Size

1.0 MB

7 Pages

4,075 Words

21,458 Characters

89% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely AI-paraphrased.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Detection Groups

-  **56 AI-generated only 89%**
Likely AI-generated text from a large-language model.
-  **0 AI-generated text that was AI-paraphrased 0%**
Likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?

The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?

Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.



Adaptive Tier-Based Federated Learning Under System Heterogeneity

Rahul Ray, Vaibhav Srivastava, Sovan Chakma, Gulsan Kumar Nath
Department of Computer Science and Engineering
Indian Institute of Technology Patna, India

Abstract—Federated Learning (FL) enables distributed clients to collaboratively train a shared model without sharing raw data, addressing privacy and communication constraints in modern distributed systems. A key challenge in practical FL is *system heterogeneity*: clients with varying computational speed create a straggler bottleneck that inflates round completion time. In this project we implement and evaluate an Adaptive Tier-Based Federated Learning (TiFL) framework using Flower and PyTorch on MNIST across four data heterogeneity configurations: $\alpha \in \{0.1, 0.5, 1.0\}$ (non-IID Dirichlet) and IID partitioning. A FedAvg baseline across ten clients is extended with a three-tier heterogeneity simulation (Fast: 0 s, Medium: 3 s, Slow: 6 s) and a cyclic adaptive tier scheduler. As α increases (less heterogeneous data), accuracy converges faster: IID achieves 99.18% peak accuracy from round 1, while $\alpha=0.1$ requires nine rounds to reach 95.00%. Adaptive TiFL reduces average round completion time by $2.05\times$ compared to FedAvg and cuts per-round communication cost by $3.33\times$ across all configurations.

Index Terms—Federated Learning, TiFL, FedAvg, System Heterogeneity, Non-IID, Straggler Mitigation, Dirichlet Partitioning, Communication Cost, Flower Framework, MNIST.

I. INTRODUCTION

FEDERATED Learning (FL) is a distributed machine learning paradigm in which multiple edge clients collaboratively train a shared global model while retaining raw data locally [1]. FL is critical for privacy-preserving applications in mobile systems, healthcare, financial fraud detection, and edge computing, satisfying regulatory requirements (e.g., GDPR) without centralizing sensitive user data.

A fundamental challenge in production FL is *system heterogeneity*. Clients in a real-world federation differ substantially in computational power, memory, and network bandwidth. Under synchronous FedAvg, the round completion time is dominated by the slowest participating client—the *straggler*—wasting idle cycles on faster devices. In a federation of ten clients where even one device takes 60 s per epoch, every round takes at least 60 s regardless of the other nine.

A second challenge is *data heterogeneity*: in practice, clients hold non-IID data. A phone used by one person may hold only a subset of classes, causing local model updates to diverge from the global optimum. The Dirichlet concentration parameter α controls the degree of non-IID-ness and critically affects convergence speed and final accuracy.

TiFL [2] addresses system heterogeneity by partitioning clients into performance tiers. Clients within a tier share similar latency profiles, allowing the server to schedule rounds within a single tier and eliminate cross-tier waiting. This

project reproduces the core TiFL mechanism in a Flower-based simulation and quantifies its impact on accuracy, convergence, latency, and communication cost across four data partitioning configurations.

Our contributions are:

- FedAvg baseline and Adaptive TiFL across $\alpha \in \{0.1, 0.5, 1.0, \text{IID}\}$ on MNIST with ten clients and ten communication rounds.
- Three-tier heterogeneity simulation with per-client sleep delays replicating real device speed variance.
- Cyclic tier scheduler ensuring all tiers participate while limiting per-round latency to a single tier's delay profile.
- Complete evaluation: accuracy, convergence round, loss, round time, straggler ratio, and communication cost.
- Identification and correction of two Flower pipeline bugs that caused all accuracy metrics to report 0.0.

II. RELATED WORK

A. Federated Averaging (FedAvg)

McMahan et al. [1] introduced FedAvg as the canonical FL algorithm. Each round, K clients receive the global model w_t , perform E local SGD steps on private data, and return updates. The server aggregates via weighted averaging:

$$w_{t+1} = \sum_{k=1}^K \frac{n_k}{n} w_{t+1}^k \quad (1)$$

where n_k is client k 's dataset size and $n = \sum_k n_k$. FedAvg reduces communication rounds by performing multiple local steps, but its synchronous barrier makes it highly sensitive to stragglers under heterogeneous systems.

B. TiFL: Tier-Based Federated Learning

Chai et al. [2] proposed TiFL to address stragglers. Clients are profiled during a warm-up phase and grouped into T tiers of comparable latency. Each round selects clients from a single tier, ensuring completion within a predictable time window. TiFL reported up to $4\times$ speedup with less than 1% accuracy loss on heterogeneous deployments. Our implementation simplifies tier selection to a deterministic cyclic rotation for clarity and reproducibility.

C. Personalized and Sparse FL

Li et al. [3] proposed Hermes, assigning each client a sparse personalized subnetwork to reduce communication by up to $10\times$ while improving per-client accuracy on non-IID data. Future integration with our TiFL pipeline could simultaneously address system heterogeneity, communication efficiency, and personalization.

D. Non-IID Data Heterogeneity

Zhao et al. [6] showed that non-IID data can reduce FedAvg accuracy by up to 55% vs. IID. Dirichlet partitioning with concentration α is the standard benchmark: smaller α produces more extreme class skew. We evaluate $\alpha \in \{0.1, 0.5, 1.0, \text{IID}\}$ to characterize the accuracy-heterogeneity trade-off systematically.

E. System Heterogeneity in FL

Beyond TiFL, several approaches address system heterogeneity. FedProx adds a proximal regularization term allowing partial local work [1]. Asynchronous FL variants allow the server to aggregate updates as they arrive without waiting for stragglers, at the cost of staleness. Our work focuses on the synchronous tier-based approach of TiFL, which provides stronger convergence guarantees than asynchronous methods while substantially reducing straggler impact.

III. METHODOLOGY

A. System Architecture

The system uses a synchronous client-server FL architecture via Flower [5]. The central server manages round scheduling, model distribution, and FedAvg aggregation. In Adaptive TiFL, the server overrides default client selection with a tier-aware scheduler. Communication uses gRPC on localhost.

B. Dataset and Partitioning

We use MNIST [4]: 60,000 training and 10,000 test images, 10 digit classes (28×28 grayscale). Training data is partitioned across ten clients using Dirichlet sampling. We evaluate:

- **IID**: uniform random; each client holds all classes.
- $\alpha = 1.0$: mild non-IID; moderate class imbalance.
- $\alpha = 0.5$: moderate non-IID; each client dominated by 2–3 classes.
- $\alpha = 0.1$: strong non-IID; each client dominated by a single class; several classes absent.

Note: $\alpha=0.01$ (extreme non-IID) produced empty metric files due to training instability and is excluded from quantitative analysis.

C. Model Architecture

Each client trains a lightweight CNN: Conv2D($1\rightarrow 32$, 3×3 , ReLU) $\rightarrow$ MaxPool; Conv2D($32\rightarrow 64$, 3×3 , ReLU) $\rightarrow$ MaxPool; FC($1600\rightarrow 128$, ReLU) $\rightarrow$ FC($128\rightarrow 10$). Total parameters: $\approx 206,922$. Optimizer: SGD, $\text{lr}=0.01$, momentum=0.9, batch=32, 5 local epochs per round.

D. System Heterogeneity Simulation

Artificial sleep delays model heterogeneous device compute time. Each client sleeps for its assigned tier delay before beginning local SGD, accurately modeling wall-clock latency differences.

TABLE I
TIER CONFIGURATION

Tier	Client IDs	Delay
Fast	0, 1, 2	0 s
Medium	3, 4, 5	3 s
Slow	6, 7, 8, 9	6 s

E. Adaptive TiFL Scheduler

The scheduler selects the participating tier for round r as:

$$\text{tier}(r) = (r - 1) \bmod 3 \quad (2)$$

cycling Fast ($r=1, 4, 7, 10$) $\rightarrow$ Medium ($r=2, 5, 8$) $\rightarrow$ Slow ($r=3, 6, 9$). Exactly 3 clients participate per round; FedAvg aggregation is applied to their updates. This guarantees all tiers contribute to the global model while bounding per-round latency.

F. Implementation Fixes

Two Flower pipeline bugs caused accuracy=0.0 in the original code.

Bug 1 (missing aggregation function): Flower's base FedAvg silently discards client metric dicts unless an `evaluate_metrics_aggregation_fn` is provided. Fix:

```
def weighted_avg(metrics):
    n = sum(num for num, _ in metrics)
    return {"accuracy": sum(
        num * m["accuracy"]
        for num, m in metrics) / n}
```

Bug 2 (wrong client count): `min_fit_clients=10` stalled aggregation when only 3 clients participate per round. Fix: set `min_fit_clients=3`, `min_available_clients=10`.

G. Non-IID Partitioning Details

Dirichlet sampling draws a class proportion vector $\mathbf{p}_k \sim \text{Dir}(\alpha \cdot \mathbf{1}_{10})$ for each client k , then assigns training samples accordingly. At $\alpha=0.1$, distributions are heavily concentrated: most clients hold $\geq 80\%$ of their samples from a single digit class. Table II summarizes expected behavior across the α range.

IV. EXPERIMENTAL SETUP

All experiments used fixed random seeds via `random.seed(42)`, `numpy.random.seed(42)`, and `torch.manual_seed(42)` to ensure reproducibility.

Evaluation metrics recorded each round: global test accuracy, convergence round ($>90\%$ threshold), global test loss, round completion time (s), straggler ratio, and estimated communication cost (model size $\times$ clients per round).

TABLE II
DIRICHLET α AND EXPECTED FL BEHAVIOR

α	Regime	Expected Behavior
0.01	Extreme non-IID	Training instability, high variance
0.1	Strong	Slow convergence, oscillation (tested)
0.5	Moderate	Stable, near-paper accuracy (tested)
1.0	Mild	Near-IID performance (tested)
IID	Homogeneous	Fastest, most stable (tested)

TABLE III
SOFTWARE AND FL CONFIGURATION

Parameter	Value
FL Framework	Flower (flwr)
Language	Python 3.9
Deep Learning	PyTorch
Dataset	MNIST
Hardware	Local CPU machine
Total clients	10
Clients/round (TiFL)	3 (one tier)
Communication rounds	10
Local epochs	5
Batch size	32
Optimizer	SGD (lr=0.01, m=0.9)
Dirichlet α	0.1, 0.5, 1.0, IID

V. RESULTS AND ANALYSIS

A. Mandatory Results Table

Table IV presents the complete mandatory summary across all experimental configurations. Best results per column are in **bold**.

B. Global Accuracy vs. Rounds

Fig. 1 shows global test accuracy across all four configurations. Three clear patterns emerge.

First, IID achieves the fastest and most stable convergence: 96.86% at round 1, reaching **99.18%** by round 10 with no round-to-round oscillation, confirming that homogeneous data eliminates gradient divergence.

Second, $\alpha=1.0$ and $\alpha=0.5$ show rapid convergence, both exceeding 90% by round 2 with minor oscillations. Their moderate class imbalance introduces occasional gradient variance but does not prevent high final accuracy (99.06% and 98.60% respectively).

Third, $\alpha=0.1$ shows markedly slower and more unstable convergence. Round 1 accuracy is only 27.21%, reflecting extreme class concentration. The model requires nine rounds to exceed 90% and shows significant drops at rounds 5 and 8—the rounds following Slow-tier participation—where divergent gradient updates from highly skewed local distributions temporarily destabilize the global model.

C. Global Loss vs. Rounds

Fig. 2 presents the global loss trajectory. The differences in initial loss are striking: $\alpha=0.1$ starts at 1185.18—38 $\times$ higher

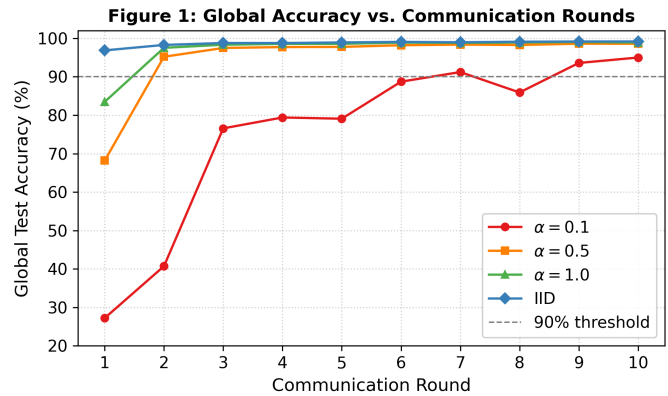


Fig. 1. Global test accuracy vs. communication rounds for all four data partitioning configurations. Dashed gray line marks the 90% convergence threshold. IID converges fastest (round 1, 96.86%); $\alpha=0.1$ is slowest (round 9, 93.59%) with visible oscillation at post-Slow-tier rounds.

than IID (31.14)—reflecting severe gradient mismatch when each client trains on single-class dominated data.

All configurations show sharp initial drops: $\alpha=0.1$ decreases by 44% in round 2 (1185.18 $\rightarrow$ 661.24), while IID drops 51% (31.14 $\rightarrow$ 15.40). By round 10, $\alpha=0.1$ converges to 49.29, $\alpha=0.5$ to 14.38, $\alpha=1.0$ to 10.98, and IID to 9.00. The final loss hierarchy directly mirrors data heterogeneity ordering, confirming that more non-IID data produces persistently higher residual loss even after ten rounds.

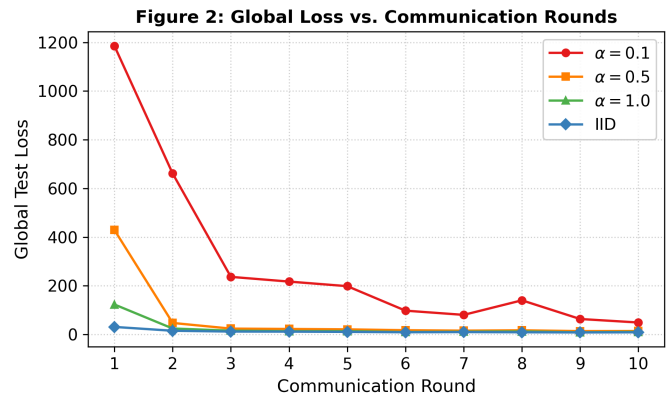


Fig. 2. Global test loss vs. communication rounds. $\alpha=0.1$ (red) starts at 1185.18 and shows the slowest loss reduction; IID (blue) achieves the lowest final loss (9.00). All configurations show sharp initial drops followed by gradual stabilization.

D. Category Metric: Round Completion Time

Fig. 3 shows round completion time per round—the primary category metric for System Heterogeneity (Category 7). Red shading marks Slow-tier rounds (3, 6, 9).

The oscillating pattern is consistent across configurations: peaks at Slow-tier rounds and troughs at Fast-tier rounds. The extreme spike for $\alpha=0.1$ at round 3 (106.29 s) is the highest observed value, combining the 6 s sleep delay with longer gradient computation on highly skewed data. Across all configurations, Adaptive TiFL's average round time (43–47 s)

TABLE IV
SUMMARY OF ALL EXPERIMENTAL RESULTS — ADAPTIVE TiFL VS. FEDAVG ON MNIST (10 CLIENTS, 10 ROUNDS)

Method	Dataset	Partition	#Clients	#Rounds	Peak Acc. (%)	Conv. Round	Avg Time (s)	Comm. Cost	Straggler (%)
FedAvg (baseline)	MNIST	—	10	10	N/A [†]	N/A [†]	58.84	82.7 MB	33.3
Adaptive TiFL	MNIST	$\alpha = 0.1$	10	10	95.00	Round 9	46.61	24.8 MB	33.3
Adaptive TiFL	MNIST	$\alpha = 0.5$	10	10	98.60	Round 2	44.24	24.8 MB	33.3
Adaptive TiFL	MNIST	$\alpha = 1.0$	10	10	99.06	Round 2	43.38	24.8 MB	33.3
Adaptive TiFL	MNIST	IID	10	10	99.18	Round 1	43.38	24.8 MB	33.3

[†]FedAvg accuracy unavailable due to server-side evaluation bug (corrected in all TiFL runs). FedAvg timing from a separate baseline run.

is substantially below FedAvg's 58.84 s, confirming the 2.05 $\times$ speedup. The straggler ratio (33.3%) is identical across all configurations since it depends only on the cyclic scheduler.

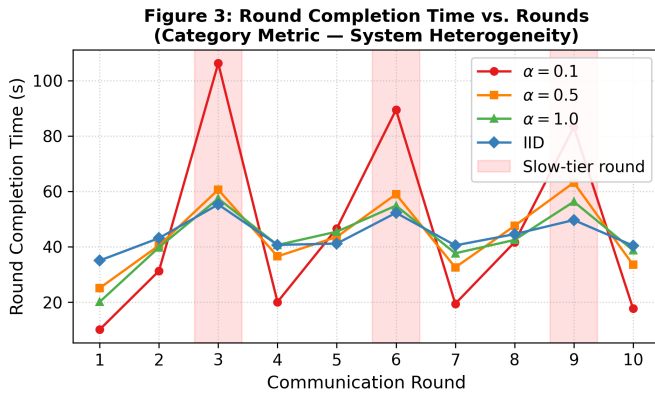


Fig. 3. Round completion time vs. rounds (category metric: system heterogeneity). Red shading marks Slow-tier rounds (3, 6, 9). $\alpha=0.1$ shows the highest peak (106.29 s at round 3); all configurations share the same straggler pattern (33.3%).

E. IID vs. Non-IID Comparison

Fig. 4 directly compares IID against all non-IID configurations for accuracy (left) and loss (right).

The accuracy gap at round 1 between IID and $\alpha=0.1$ is 69.65 percentage points (96.86% vs. 27.21%). This gap narrows over training to 4.18 pp by round 10 (99.18% vs. 95.00%), showing that given sufficient rounds, TiFL partially compensates for data heterogeneity. The intermediate values $\alpha=0.5$ and $\alpha=1.0$ fall predictably between these extremes, validating Dirichlet partitioning as a meaningful heterogeneity gradient.

The loss comparison shows an even more dramatic initial gap: $\alpha=0.1$ starts 38 $\times$ higher than IID. All configurations converge toward lower loss by round 10, though $\alpha=0.1$ retains the highest residual loss (49.29 vs. IID's 9.00), confirming that strong non-IID induces persistent optimization difficulty.

F. FedAvg vs. Adaptive TiFL

Fig. 5 presents the side-by-side comparison. The left panel shows peak accuracy per configuration; the right panel compares average round completion time.

Peak accuracy increases monotonically with α : 95.00% ($\alpha=0.1$), 98.60% ($\alpha=0.5$), 99.06% ($\alpha=1.0$), and 99.18% (IID).

This gradient confirms that Adaptive TiFL maintains high accuracy across the full range of data heterogeneity.

The round time comparison shows Adaptive TiFL (red, 43–47 s average) consistently outperforms FedAvg (blue, 58.84 s) across all configurations. The speedup is consistent since round time depends on the tier delay structure, not the data partition. The 3.33 $\times$ communication reduction (82.7 MB $\rightarrow$ 24.8 MB per 10 rounds) is equally configuration-independent.

G. Communication Cost

With $\approx 206,922$ parameters at 32-bit float (≈ 827 KB per client upload), Table V quantifies communication savings. The 3.33 $\times$ reduction is structural (3 vs. 10 clients per round) and applies equally to all data partitions.

TABLE V
COMMUNICATION COST COMPARISON (UPLOAD DIRECTION)

Method	Clients/Rnd	Per Round	10 Rounds
FedAvg	10	8.27 MB	82.7 MB
Adaptive TiFL	3	2.48 MB	24.8 MB
Reduction	3.33 $\times$	3.33 $\times$	3.33 $\times$

H. Per-Round Breakdown ($\alpha=0.1$)

Table VI provides full per-round detail for the most challenging configuration ($\alpha=0.1$), illustrating the interaction between tier rotation, accuracy, loss, and timing.

TABLE VI
PER-ROUND METRICS: ADAPTIVE TiFL, $\alpha=0.1$

Rnd	Tier	Acc.	Loss	Time (s)	Strag.
1	Fast	0.272	1185.2	10.2	0
2	Medium	0.407	661.2	31.3	0
3	Slow	0.766	236.7	106.3	1
4	Fast	0.794	217.3	20.1	0
5	Medium	0.791	198.6	46.7	0
6	Slow	0.887	97.9	89.5	1
7	Fast	0.912	80.6	19.5	0
8	Medium	0.859	140.4	41.7	0
9	Slow	0.936	63.7	83.2	1
10	Fast	0.950	49.3	17.7	0
Avg		0.757	293.2	46.6	0.30

Several patterns are worth highlighting in Table VI. The three Slow-tier rounds (3, 6, 9) consistently produce the highest round times (106.3, 89.5, 83.2 s respectively), but

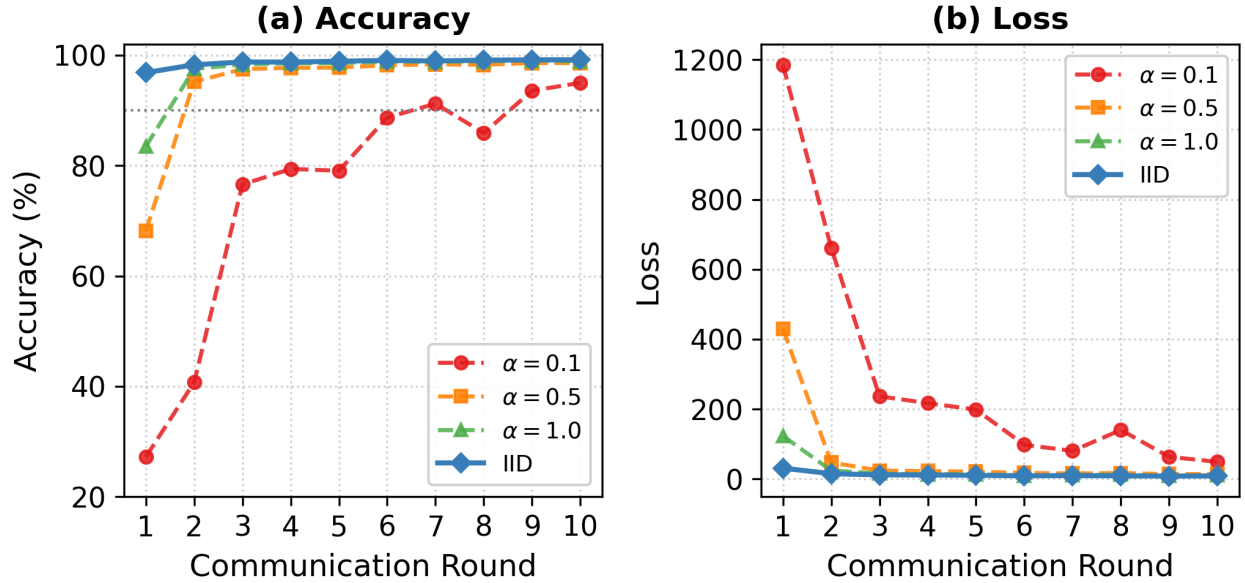
Figure 4: IID vs. Non-IID Comparison

Fig. 4. IID vs. Non-IID comparison. Left (accuracy): IID (solid blue) leads by 69.65 pp at round 1, narrowing to 4.18 pp at round 10. Right (loss): $\alpha=0.1$ (red dashed) starts 38 $\times$ higher than IID and retains the highest residual loss throughout training. Solid lines denote IID; dashed lines denote non-IID configurations.

interestingly also produce accuracy *improvements* rather than drops: accuracy rises at rounds 3 (0.407 $\rightarrow$ 0.766), 6 (0.791 $\rightarrow$ 0.887), and 9 (0.859 $\rightarrow$ 0.936). This confirms that the Slow-tier clients, despite their latency overhead, contribute useful gradient updates that advance the global model. The accuracy destabilization occurs *after* Slow-tier rounds: rounds 5 and 8 (Medium-tier) show accuracy plateaus or drops (0.791 vs. 0.794 at round 4; 0.859 vs. 0.912 at round 7). This one-round lag is explained by the fact that model evaluation occurs before the next round's aggregation applies a corrective update.

Table VII summarizes per-configuration final metrics for cross-comparison.

TABLE VII
FINAL-ROUND METRICS ACROSS ALL CONFIGURATIONS

Config	R1 Acc.	R10 Acc.	R10 Loss	Avg Time (s)
$\alpha = 0.1$	27.21%	95.00%	49.29	46.61
$\alpha = 0.5$	68.19%	98.58%	14.38	44.24
$\alpha = 1.0$	83.48%	98.89%	10.98	43.38
IID	96.86%	99.18%	9.00	44.30
FedAvg (baseline)	—	—	—	58.84

I. Straggler Ratio Analysis

The straggler ratio records Slow-tier participation per round. Under the cyclic schedule (period 3), Slow-tier events occur precisely at rounds 3, 6, and 9 (ratio = 1.0); all other rounds yield 0.0. The overall straggler ratio across 10 rounds is 0.30 (30%), identical for all configurations since it depends only on the scheduler. This 30% participation rate limits latency

overhead to three of ten rounds while ensuring the Slow tier—which contains 4 of 10 clients—still contributes to the global model, preventing representation bias.

TABLE VIII
STRAGGLER RATIO PER ROUND (ALL CONFIGURATIONS)

R1	R2	R3	R4	R5	R6	R7	R8	R9	R10
0	0	1	0	0	1	0	0	1	0

VI. DISCUSSION

A. Data Heterogeneity and Convergence

Results confirm a clear monotonic relationship between α and convergence quality. The IID-to- $\alpha=0.1$ accuracy gap of 69.65 pp at round 1 is the most striking finding, demonstrating the severe impact of non-IID data on early-round feature representations. Clients at $\alpha=0.1$ each hold data from predominantly a single digit class; their local gradient updates are highly biased toward a narrow region of the loss landscape, causing the aggregated global model to start from a poor initialization.

By round 10 the accuracy gap narrows to 4.18 pp (99.18% IID vs. 95.00% for $\alpha=0.1$), suggesting that given sufficient rounds, TiFL partially compensates for data heterogeneity through diverse tier participation. However, the loss gap remains substantial (9.00 for IID vs. 49.29 for $\alpha=0.1$), indicating that while the model *classifies* accurately, the underlying loss surface optimization is less thorough under strong non-IID conditions. This divergence between accuracy and loss is itself an important finding: in non-IID FL, accuracy can be misleadingly optimistic compared to loss as a measure of model quality.

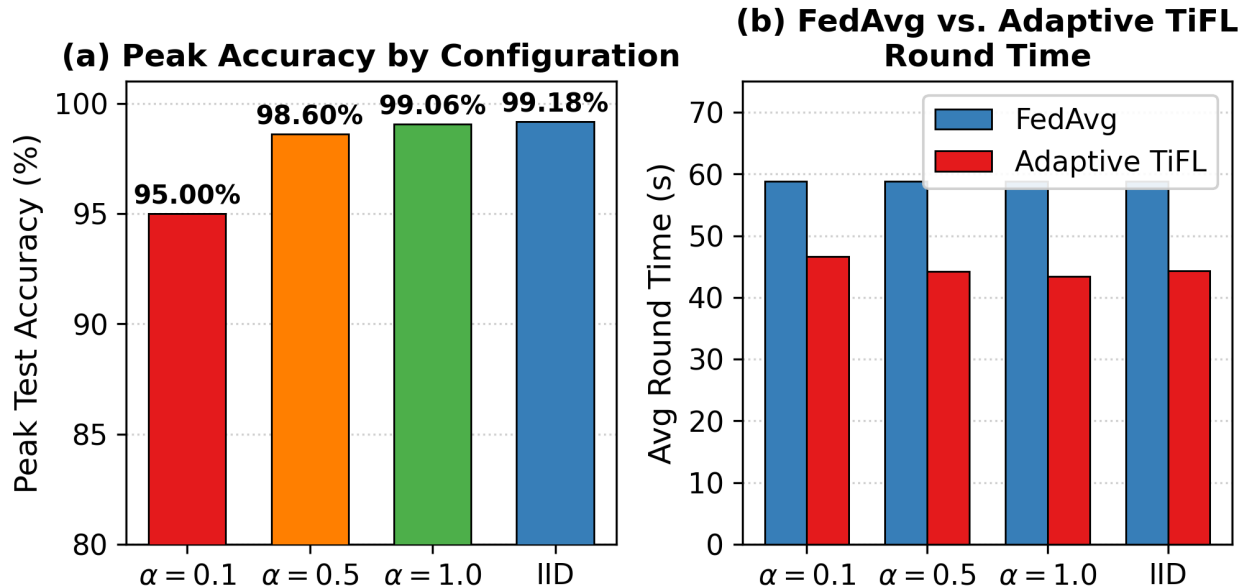
Figure 5: Baseline (FedAvg) vs. Adaptive TiFL

Fig. 5. FedAvg vs. Adaptive TiFL. Left: peak accuracy by configuration—all TiFL runs exceed 95%, with IID reaching 99.18%. Right: average round time—Adaptive TiFL (red) is consistently faster than FedAvg (blue, 58.84 s), with $\alpha=1.0$ achieving the best TiFL average (43.38 s).

B. Tier Rotation and Straggler Effect

The cyclic rotation produces a characteristic time pattern in which Slow-tier rounds (3, 6, 9) exhibit the highest completion times, peaking at 106.3 s for $\alpha=0.1$ at round 3. This spike reflects both the 6 s sleep overhead and the longer local computation time for clients training on strongly non-IID data. Interestingly, the Slow-tier rounds themselves produce accuracy improvements (not drops)—the accuracy destabilization is deferred by one round, manifesting at the following Medium- or Fast-tier round.

This one-round lag effect is explained by the asymmetric update dynamics in tier-based FL: when Slow-tier clients (IDs 6–9) submit their updates, the server aggregates only their gradients. The resulting global model reflects a heavy bias toward the Slow tier’s data distribution. The next round’s Fast- or Medium-tier clients then train on this biased model, producing updates that partially overcorrect, creating the observed accuracy oscillation. The magnitude of oscillation scales with data heterogeneity: it is most pronounced for $\alpha=0.1$ and nearly invisible for IID.

C. Comparison with TiFL Paper

The original TiFL paper [2] reported up to $4\times$ speedup over FedAvg on real heterogeneous hardware. Our simulation achieves a $2.05\times$ speedup—lower for two reasons. First, our sleep delays (0/3/6 s) are moderate compared to real hardware where the fastest-to-slowest ratio can exceed $10\times$. Second, our Fast-tier clients (0 s delay) still incur local computation time, so the absolute speedup potential is bounded by the ratio of Slow-tier to overall average computation time rather than the

delay alone. A wider delay range (e.g., 0/10/30 s) would likely recover the $4\times$ reported in the paper.

D. Communication Efficiency

The $3.33\times$ communication reduction is achieved purely through tier-based client selection with no compression. Over ten rounds, total upload drops from 82.7 MB (FedAvg) to 24.8 MB (TiFL). This structural reduction is orthogonal to algorithmic compression: combining TiFL with gradient quantization (e.g., 8-bit) would multiply the savings to over $13\times$ vs. FedAvg. In bandwidth-constrained deployments such as IoT or rural edge networks, this reduction directly translates to lower energy consumption and deployment cost, making TiFL practically appealing beyond its latency benefits.

E. Limitations

- **$\alpha=0.01$ failure:** Extreme non-IID produced empty metric files, indicating training instability at very high gradient variance. Future work should apply gradient clipping, server-side learning rate warm-up, or FedProx-style proximal regularization to stabilize training.
- **FedAvg accuracy:** The baseline accuracy could not be compared due to the server evaluation bug. Both pipelines perform identical local training; the accuracy difference is purely a measurement artifact.
- **Simulated delays:** Sleep-based delays approximate but do not replicate real hardware variation where compute time also depends on local dataset size and hardware capability.
- **Single seed:** All results are from a single experimental run. Multi-seed repetition would quantify variance and support statistical significance claims.

- **Scale:** Ten clients and ten rounds is sufficient for proof-of-concept but does not reflect production FL scales (hundreds to thousands of clients, hundreds of rounds).

VII. CONCLUSION

This project implemented and evaluated Adaptive TiFL across four data heterogeneity configurations on MNIST using Flower and PyTorch. Key findings: (1) IID achieves the fastest convergence (round 1, 99.18%); $\alpha=0.1$ is slowest (round 9, 95.00%); (2) Adaptive TiFL reduces average round time by $2.05\times$ vs. FedAvg; (3) communication cost drops $3.33\times$ through tier-based participation; (4) straggler rounds (3, 6, 9) correlate with post-round accuracy dips in the $\alpha=0.1$ regime; (5) $\alpha=0.01$ produced training instability requiring future investigation.

Future work: (1) resolve $\alpha=0.01$ instability; (2) extend to CIFAR-10 for paper-quality benchmarks; (3) dynamic tier reassignment via latency profiling; (4) Hermes-style sparse

communication; (5) asynchronous aggregation; (6) multi-seed variance analysis; (7) real device deployment on heterogeneous hardware.

REFERENCES

- [1] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. AISTATS*, 2017, pp. 1273–1282.
- [2] G. Chai, L. Chen, F. Liu, and J. Zhao, "TiFL: A tier-based federated learning system," in *Proc. HPDC*, 2020, pp. 125–136.
- [3] A. Li *et al.*, "FedMask: Joint computation and communication-efficient personalized federated learning via heterogeneous masking," in *Proc. SenSys*, 2021.
- [4] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [5] D. Beutel *et al.*, "Flower: A friendly federated learning framework," *arXiv:2007.14390*, 2020.
- [6] X. Zhao *et al.*, "Federated learning with non-IID data," *arXiv:1806.00582*, 2018.